

Cours Complet Tkinter pour Débutants

Table des matières

1. Les bases - Première fenêtre
2. Les widgets essentiels
3. La gestion des événements
4. Le positionnement (geometry managers)
5. Widgets intermédiaires
6. Organisation et frames
7. Menus et boîtes de dialogue
8. Canvas et projets

Chapitre 1 : Les bases - Première fenêtre

Introduction à Tkinter

Tkinter (Tk interface) est la bibliothèque graphique standard de Python. Elle permet de créer des interfaces utilisateur (GUI - Graphical User Interface) de manière simple et efficace. Tkinter est inclus par défaut avec Python, vous n'avez donc rien à installer.

Votre première fenêtre

```
```python
```

```
import tkinter as tk
```

```
Créer la fenêtre principale
```

```
fenetre = tk.Tk()
```

```
Lancer la boucle d'événements
```

```
fenetre.mainloop()
```

```
...
```

```
Explications :
```

```
- `import tkinter as tk` : importe la bibliothèque et lui donne l'alias "tk" pour simplifier l'écriture
```

```
- `tk.Tk()` : crée la fenêtre principale de l'application
```

```
- `mainloop()` : lance la boucle d'événements qui maintient la fenêtre ouverte et réactive aux interactions
```

```
Personnaliser la fenêtre
```

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
# Définir le titre de la fenêtre
```

```
fenetre.title("Ma première application Tkinter")
```

```
# Définir la taille de la fenêtre (largeur x hauteur)
```

```
fenetre.geometry("400x300")
```

```
# Définir la couleur de fond
```

```
fenetre.configure(bg="lightblue")
```

```
# Empêcher le redimensionnement
```

```
fenetre.resizable(False, False)
```

```
fenetre.mainloop()
```

```
...
```

****Méthodes importantes :****

- ``title(texte)`` : définit le titre affiché dans la barre de titre
- ``geometry("LxH")`` : définit les dimensions en pixels (largeur x hauteur)
- ``configure(bg=couleur)`` : change la couleur de fond
- ``resizable(width, height)`` : autorise (True) ou interdit (False) le redimensionnement

Positionner la fenêtre à l'écran

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Fenêtre positionnée")
```

```
Positionner la fenêtre (largeur x hauteur + position_x + position_y)
```

```
fenetre.geometry("400x300+100+50")
```

```
fenetre.mainloop()
```

```
...
```

La syntaxe complète est : ``"LARGEURxHAUTEUR+X+Y"`` où X et Y sont les coordonnées du coin supérieur gauche.

```
Centrer la fenêtre
```

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Fenêtre centrée")
```

```
largeur_fenetre = 400
```

```
hauteur_fenetre = 300
```

```
# Obtenir les dimensions de l'écran
```

```
largeur_ecran = fenetre.winfo_screenwidth()
```

```
hauteur_ecran = fenetre.winfo_screenheight()
```

```
# Calculer la position x et y pour centrer
```

```
x = (largeur_ecran - largeur_fenetre) // 2
```

```
y = (hauteur_ecran - hauteur_fenetre) // 2
```

```
fenetre.geometry(f"{largeur_fenetre}x{hauteur_fenetre}+{x}+{y}")
```

```
fenetre.mainloop()
```

```
...
```

```
### Exercices Chapitre 1
```

```
**Exercice 1.1 :** Créez une fenêtre de 500x400 pixels avec le titre "Mon Portfolio" et une couleur de fond verte pâle.
```

****Exercice 1.2 : **** Créez une fenêtre centrée sur l'écran, de dimensions 300x200, avec le titre "Calculatrice" et interdisez son redimensionnement.

****Exercice 1.3 : **** Créez une fenêtre qui s'affiche en haut à droite de l'écran.

Chapitre 2 : Les widgets essentiels

Qu'est-ce qu'un widget ?

Un widget est un élément graphique de l'interface (bouton, zone de texte, étiquette, etc.). Tkinter propose de nombreux widgets pour construire vos applications.

Le widget Label (étiquette)

Le Label permet d'afficher du texte ou une image.

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Widget Label")
```

```
fenetre.geometry("400x200")
```

```
Créer un label simple
```

```
etiquette1 = tk.Label(fenetre, text="Bonjour, bienvenue dans Tkinter !")
```

```
etiquette1.pack()
```

# Label avec personnalisation

```
etiquette2 = tk.Label(
 fenetre,
 text="Texte personnalisé",
 font=("Arial", 16, "bold"),
 fg="blue",
 bg="yellow",
 padx=20,
 pady=10
)
etiquette2.pack()
```

fenetre.mainloop()

...

**\*\*Paramètres principaux du Label :\*\***

- `text` : le texte à afficher
- `font` : police (nom, taille, style)
- `fg` (foreground) : couleur du texte
- `bg` (background) : couleur de fond
- `padx`, `pady` : espacement interne horizontal et vertical
- `width`, `height` : largeur et hauteur en caractères

**### Le widget Button (bouton)**

Le Button crée un bouton cliquable.

```
```python
import tkinter as tk

fenetre = tk.Tk()
fenetre.title("Widget Button")
fenetre.geometry("300x200")

# Bouton simple
bouton1 = tk.Button(fenetre, text="Cliquez-moi !")
bouton1.pack(pady=10)

# Bouton personnalisé
bouton2 = tk.Button(
    fenetre,
    text="Bouton stylé",
    font=("Courier", 12),
    bg="green",
    fg="white",
    padx=20,
    pady=10,
    relief="raised",
    bd=5
)
bouton2.pack(pady=10)

fenetre.mainloop()
```
```

**\*\*Paramètres supplémentaires du Button :\*\***

- `relief` : style du relief ("flat", "raised", "sunken", "groove", "ridge")
- `bd` (borderwidth) : épaisseur de la bordure
- `state` : état du bouton ("normal", "active", "disabled")
- `cursor` : forme du curseur au survol

**### Le widget Entry (champ de saisie)**

L'Entry permet à l'utilisateur d'entrer du texte sur une ligne.

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Widget Entry")
```

```
fenetre.geometry("400x200")
```

```
# Label descriptif
```

```
label = tk.Label(fenetre, text="Entrez votre nom :")
```

```
label.pack(pady=10)
```

```
# Champ de saisie
```

```
entree = tk.Entry(fenetre, width=30, font=("Arial", 12))
```

```
entree.pack(pady=5)
```

```
# Entry avec texte par défaut
```

```
entree2 = tk.Entry(fenetre, width=30)
```

```
entree2.insert(0, "Texte par défaut")
```



```
entree2.pack(pady=5)
```

```
# Entry pour mot de passe
```

```
label_mdp = tk.Label(fenetre, text="Mot de passe :")
```

```
label_mdp.pack(pady=10)
```

```
entree_mdp = tk.Entry(fenetre, show="*", width=30)
```

```
entree_mdp.pack(pady=5)
```

```
fenetre.mainloop()
```

```
...
```

```
**Méthodes importantes de l'Entry :**
```

```
- `insert(position, texte)` : insère du texte à une position (0 = début)
```

```
- `get()` : récupère le texte saisi
```

```
- `delete(debut, fin)` : supprime le texte entre deux positions
```

```
- `show` : caractère à afficher (utile pour les mots de passe)
```

```
### Combiner les trois widgets
```

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Formulaire simple")
```

```
fenetre.geometry("400x300")
```

```
Titre
```

```
titre = tk.Label(
```

```
fenetre,
text="Formulaire d'inscription",
font=("Arial", 18, "bold"),
fg="darkblue"
)
titre.pack(pady=20)
```

```
Nom
```

```
label_nom = tk.Label(fenetre, text="Nom :", font=("Arial", 12))
label_nom.pack()
entree_nom = tk.Entry(fenetre, width=30)
entree_nom.pack(pady=5)
```

```
Prénom
```

```
label_prenom = tk.Label(fenetre, text="Prénom :", font=("Arial", 12))
label_prenom.pack()
entree_prenom = tk.Entry(fenetre, width=30)
entree_prenom.pack(pady=5)
```

```
Email
```

```
label_email = tk.Label(fenetre, text="Email :", font=("Arial", 12))
label_email.pack()
entree_email = tk.Entry(fenetre, width=30)
entree_email.pack(pady=5)
```

```
Bouton
```

```
bouton_valider = tk.Button(
 fenetre,
```

```
text="S'inscrire",
bg="green",
fg="white",
font=("Arial", 12, "bold"),
padx=20,
pady=5
)
bouton_valider.pack(pady=20)

fenetre.mainloop()
...
```

## ### Exercices Chapitre 2

**\*\*Exercice 2.1 :\*\*** Créez une fenêtre avec un Label affichant "Quelle est votre couleur préférée ?" et un Entry pour la réponse.

**\*\*Exercice 2.2 :\*\*** Créez un formulaire de connexion avec deux Entry (identifiant et mot de passe) et un bouton "Se connecter".

**\*\*Exercice 2.3 :\*\*** Créez une interface avec trois Labels de couleurs différentes et trois boutons également de couleurs différentes.

---

## ## Chapitre 3 : La gestion des événements

### ### Introduction aux événements

Un événement est une action de l'utilisateur (clic, saisie clavier, etc.). Pour rendre votre interface interactive, vous devez gérer ces événements avec des fonctions callback.

### ### Les fonctions callback

Une fonction callback est une fonction appelée automatiquement lorsqu'un événement se produit.

```
```python
```

```
import tkinter as tk
```

```
def dire_bonjour():  
    print("Bonjour !")
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Callback simple")
```

```
fenetre.geometry("300x150")
```

```
bouton = tk.Button(fenetre, text="Dire bonjour", command=dire_bonjour)
```

```
bouton.pack(pady=50)
```

```
fenetre.mainloop()
```

```
```
```

**\*\*Important :\*\*** Notez qu'on écrit `command=dire_bonjour`` et non `command=dire_bonjour()`. Sans les parenthèses, on passe la fonction elle-même ; avec les parenthèses, on exécuterait la fonction immédiatement.

### ### Modifier des widgets depuis un callback

```

```python
import tkinter as tk

def changer_texte():
    label.config(text="Le texte a été modifié !")

fenetre = tk.Tk()
fenetre.title("Modifier un widget")
fenetre.geometry("300x150")

label = tk.Label(fenetre, text="Texte original", font=("Arial", 14))
label.pack(pady=20)

bouton = tk.Button(fenetre, text="Changer le texte", command=changer_texte)
bouton.pack(pady=10)

fenetre.mainloop()
```

```

La méthode `config()` permet de modifier les propriétés d'un widget après sa création.

### Récupérer des valeurs depuis un Entry

```

```python
import tkinter as tk

def afficher_nom():

```

```
nom = entree_nom.get()

label_resultat.config(text=f"Bonjour {nom} !")
```

```
fenetre = tk.Tk()

fenetre.title("Récupérer une valeur")

fenetre.geometry("400x200")
```

```
label_instruction = tk.Label(fenetre, text="Entrez votre nom :", font=("Arial", 12))

label_instruction.pack(pady=10)
```

```
entree_nom = tk.Entry(fenetre, width=30)

entree_nom.pack(pady=5)
```

```
bouton = tk.Button(fenetre, text="Valider", command=afficher_nom)

bouton.pack(pady=10)
```

```
label_resultat = tk.Label(fenetre, text="", font=("Arial", 14, "bold"), fg="blue")

label_resultat.pack(pady=10)
```

```
fenetre.mainloop()
```

```
...
```

```
### Passer des paramètres à une fonction callback
```

Pour passer des paramètres, on utilise des fonctions lambda ou des fonctions partielles.

```
```python

import tkinter as tk
```

```
def changer_couleur(couleur):
 fenetre.config(bg=couleur)

fenetre = tk.Tk()
fenetre.title("Passer des paramètres")
fenetre.geometry("300x200")

Utilisation de lambda pour passer un paramètre
bouton_rouge = tk.Button(
 fenetre,
 text="Rouge",
 command=lambda: changer_couleur("red")
)
bouton_rouge.pack(pady=5)

bouton_vert = tk.Button(
 fenetre,
 text="Vert",
 command=lambda: changer_couleur("green")
)
bouton_vert.pack(pady=5)

bouton_bleu = tk.Button(
 fenetre,
 text="Bleu",
 command=lambda: changer_couleur("blue")
)
```

```
bouton_bleu.pack(pady=5)
```

```
fenetre.mainloop()
```

```
...
```

```
Exemple complet : Calculatrice simple
```

```
```python
```

```
import tkinter as tk
```

```
def calculer():
```

```
    try:
```

```
        nombre1 = float(entree1.get())
```

```
        nombre2 = float(entree2.get())
```

```
        operation = operation_var.get()
```

```
        if operation == "addition":
```

```
            resultat = nombre1 + nombre2
```

```
        elif operation == "soustraction":
```

```
            resultat = nombre1 - nombre2
```

```
        elif operation == "multiplication":
```

```
            resultat = nombre1 * nombre2
```

```
        elif operation == "division":
```

```
            if nombre2 != 0:
```

```
                resultat = nombre1 / nombre2
```

```
            else:
```

```
                resultat = "Erreur : division par zéro"
```



```
        label_resultat.config(text=f"Résultat : {resultat}")
    except ValueError:
        label_resultat.config(text="Erreur : nombres invalides")

fenetre = tk.Tk()
fenetre.title("Calculatrice simple")
fenetre.geometry("400x300")

# Titre
titre = tk.Label(fenetre, text="Calculatrice", font=("Arial", 16, "bold"))
titre.pack(pady=10)

# Premier nombre
tk.Label(fenetre, text="Nombre 1 :").pack()
entree1 = tk.Entry(fenetre, width=20)
entree1.pack(pady=5)

# Deuxième nombre
tk.Label(fenetre, text="Nombre 2 :").pack()
entree2 = tk.Entry(fenetre, width=20)
entree2.pack(pady=5)

# Variable pour stocker l'opération
operation_var = tk.StringVar(value="addition")

# Boutons d'opération
tk.Label(fenetre, text="Opération :").pack(pady=5)
frame_operations = tk.Frame(fenetre)
```

```
frame_operations.pack()
```

```
operations = [("Addition", "addition"), ("Soustraction", "soustraction"),  
              ("Multiplication", "multiplication"), ("Division", "division")]
```

```
for texte, valeur in operations:
```

```
    tk.Radiobutton(  
        frame_operations,  
        text=texte,  
        variable=operation_var,  
        value=valeur  
    ).pack(side="left", padx=5)
```

```
# Bouton calculer
```

```
bouton_calculer = tk.Button(  
    fenetre,  
    text="Calculer",  
    command=calculer,  
    bg="green",  
    fg="white",  
    font=("Arial", 12, "bold")  
)  
bouton_calculer.pack(pady=10)
```

```
# Résultat
```

```
label_resultat = tk.Label(fenetre, text="", font=("Arial", 14, "bold"), fg="blue")  
label_resultat.pack(pady=10)
```

```
fenetre.mainloop()
```

```
...
```

Exercices Chapitre 3

****Exercice 3.1 : **** Créez un compteur avec un Label affichant un nombre et deux boutons "+" et "-" pour l'incrémenter ou le décrémenter.

****Exercice 3.2 : **** Créez un convertisseur Celsius vers Fahrenheit. L'utilisateur entre une température en Celsius, clique sur "Convertir", et le résultat s'affiche (formule : $F = C \times 9/5 + 32$).

****Exercice 3.3 : **** Créez un générateur de message de bienvenue qui demande le prénom et le nom, puis affiche "Bienvenue [Prénom] [Nom] !" dans un Label.

```
---
```

Chapitre 4 : Le positionnement (geometry managers)

Introduction aux gestionnaires de géométrie

Tkinter propose trois gestionnaires pour positionner les widgets : ``pack()``, ``grid()``, et ``place()``. Vous ne pouvez pas mélanger `pack()` et `grid()` dans le même conteneur parent.

Le gestionnaire `pack()`

``pack()`` est le plus simple. Il empile les widgets verticalement ou horizontalement.

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()

fenetre.title("Gestionnaire pack")

fenetre.geometry("400x300")

Empilement vertical (par défaut)

tk.Label(fenetre, text="Widget 1", bg="red", fg="white").pack()

tk.Label(fenetre, text="Widget 2", bg="green", fg="white").pack()

tk.Label(fenetre, text="Widget 3", bg="blue", fg="white").pack()

fenetre.mainloop()

...

```

**\*\*Paramètres de pack() :\*\***

```
```python

import tkinter as tk


fenetre = tk.Tk()

fenetre.title("pack() avec paramètres")

fenetre.geometry("500x400")


# side : côté où placer le widget

tk.Label(fenetre, text="TOP", bg="red").pack(side="top")

tk.Label(fenetre, text="BOTTOM", bg="blue").pack(side="bottom")

tk.Label(fenetre, text="LEFT", bg="green").pack(side="left")

tk.Label(fenetre, text="RIGHT", bg="yellow").pack(side="right")

```

fill : remplit l'espace disponible

```
tk.Label(fenetre, text="FILL X", bg="orange").pack(fill="x")
```

expand : prend tout l'espace disponible

```
tk.Label(fenetre, text="EXPAND", bg="purple").pack(expand=True)
```

padx, pady : marges externes

```
tk.Label(fenetre, text="AVEC PADDING", bg="cyan").pack(padx=20, pady=20)
```

ipadx, ipady : marges internes

```
tk.Label(fenetre, text="AVEC IPADDING", bg="pink").pack(ipadx=30, ipady=10)
```

```
fenetre.mainloop()
```

...

****Options principales :****

- `side` : "top" (défaut), "bottom", "left", "right"

- `fill` : "none" (défaut), "x", "y", "both"

- `expand` : True ou False

- `padx`, `pady` : marges externes

- `ipadx`, `ipady` : marges internes

- `anchor` : point d'ancrage ("n", "s", "e", "w", "center", etc.)

Le gestionnaire grid()

`grid()` organise les widgets dans une grille (lignes et colonnes). C'est le plus utilisé pour des interfaces complexes.

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Gestionnaire grid")
```

```
fenetre.geometry("400x300")
```

```
Créer une grille simple
```

```
tk.Label(fenetre, text="Ligne 0, Colonne 0", bg="red").grid(row=0, column=0)
```

```
tk.Label(fenetre, text="Ligne 0, Colonne 1", bg="green").grid(row=0, column=1)
```

```
tk.Label(fenetre, text="Ligne 1, Colonne 0", bg="blue").grid(row=1, column=0)
```

```
tk.Label(fenetre, text="Ligne 1, Colonne 1", bg="yellow").grid(row=1, column=1)
```

```
fenetre.mainloop()
```

```
```
```

```
**Exemple de formulaire avec grid() :**
```

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Formulaire avec grid")
```

```
fenetre.geometry("400x250")
```

```
Labels à gauche, Entries à droite
```

```
tk.Label(fenetre, text="Nom :").grid(row=0, column=0, sticky="e", padx=10, pady=10)
```

```
tk.Entry(fenetre, width=30).grid(row=0, column=1, padx=10, pady=10)
```

```
tk.Label(fenetre, text="Prénom :").grid(row=1, column=0, sticky="e", padx=10, pady=10)
```

```
tk.Entry(fenetre, width=30).grid(row=1, column=1, padx=10, pady=10)
```

```
tk.Label(fenetre, text="Email :").grid(row=2, column=0, sticky="e", padx=10, pady=10)
```

```
tk.Entry(fenetre, width=30).grid(row=2, column=1, padx=10, pady=10)
```

```
tk.Label(fenetre, text="Téléphone :").grid(row=3, column=0, sticky="e", padx=10, pady=10)
```

```
tk.Entry(fenetre, width=30).grid(row=3, column=1, padx=10, pady=10)
```

```
Bouton sur deux colonnes
```

```
tk.Button(fenetre, text="Valider", bg="green", fg="white").grid(
```

```
 row=4, column=0, colspan=2, pady=20
```

```
)
```

```
fenetre.mainloop()
```

```
...
```

```
Paramètres de grid() :
```

```
- `row` : numéro de ligne (commence à 0)
```

```
- `column` : numéro de colonne (commence à 0)
```

```
- `rowspan` : nombre de lignes occupées
```

```
- `colspan` : nombre de colonnes occupées
```

```
- `sticky` : alignement ("n", "s", "e", "w" ou combinaisons comme "ew", "nsew")
```

```
- `padx`, `pady` : marges externes
```

```
- `ipadx`, `ipady` : marges internes
```

```
Configurer les colonnes et lignes :
```

```
```python
import tkinter as tk

fenetre = tk.Tk()
fenetre.title("Configuration grid")
fenetre.geometry("500x300")

# Configurer les colonnes pour qu'elles s'étendent
fenetre.columnconfigure(0, weight=1)
fenetre.columnconfigure(1, weight=2)
fenetre.columnconfigure(2, weight=1)

# Configurer les lignes
fenetre.rowconfigure(0, weight=1)
fenetre.rowconfigure(1, weight=1)

tk.Label(fenetre, text="Col 0", bg="red").grid(row=0, column=0, sticky="nsew")
tk.Label(fenetre, text="Col 1 (plus large)", bg="green").grid(row=0, column=1, sticky="nsew")
tk.Label(fenetre, text="Col 2", bg="blue").grid(row=0, column=2, sticky="nsew")

tk.Label(fenetre, text="Ligne 1", bg="yellow").grid(row=1, column=0, columnspan=3,
sticky="nsew")

fenetre.mainloop()
...

### Le gestionnaire place()
```


`place()` permet un positionnement absolu ou relatif. Moins utilisé car moins flexible.

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Gestionnaire place")
```

```
fenetre.geometry("500x400")
```

```
Positionnement absolu (en pixels)
```

```
tk.Label(fenetre, text="Position (50, 50)", bg="red").place(x=50, y=50)
```

```
tk.Label(fenetre, text="Position (200, 100)", bg="green").place(x=200, y=100)
```

```
Positionnement relatif (en pourcentage)
```

```
tk.Label(fenetre, text="Centre", bg="blue").place(relx=0.5, rely=0.5, anchor="center")
```

```
Dimensions relatives
```

```
tk.Button(fenetre, text="50% de largeur").place(relx=0.5, rely=0.8, relwidth=0.5,
anchor="center")
```

```
fenetre.mainloop()
```

```
```
```

```
**Paramètres de place() :**
```

```
- `x`, `y` : position absolue en pixels
```

```
- `relx`, `rely` : position relative (0.0 à 1.0)
```

```
- `width`, `height` : dimensions absolues
```

- ``relwidth``, ``relheight`` : dimensions relatives
- ``anchor`` : point d'ancrage

Comparaison des gestionnaires

| Gestionnaire | Avantages | Inconvénients | Utilisation |
|--------------------------|--------------------|--|--------------------------------|
| <code>**pack()**</code> | Simple, rapide | Limité pour layouts complexes | Interfaces simples |
| <code>**grid()**</code> | Flexible, puissant | Plus verbeux | Formulaires, layouts complexes |
| <code>**place()**</code> | Contrôle absolu | Rigide, problèmes de redimensionnement | Rarement utilisé |

Exercices Chapitre 4

****Exercice 4.1 : **** Créez une calculatrice avec `grid()` : disposez les chiffres 0-9 en grille, avec les opérateurs sur le côté et un affichage en haut.

****Exercice 4.2 : **** Créez un formulaire d'inscription complet avec `grid()` contenant : nom, prénom, email, mot de passe, confirmation du mot de passe, et un bouton "S'inscrire".

****Exercice 4.3 : **** Créez une interface avec trois zones de couleurs différentes en utilisant `pack()` avec les options `fill` et `expand`.

Chapitre 5 : Widgets intermédiaires

Le widget Checkbutton (case à cocher)

Le Checkbutton permet de sélectionner ou désélectionner une option.

```
```python
import tkinter as tk

fenetre = tk.Tk()
fenetre.title("Checkbutton")
fenetre.geometry("300x200")

Variables pour stocker les états
var1 = tk.BooleanVar()
var2 = tk.BooleanVar()
var3 = tk.BooleanVar()

Création des checkbuttons
check1 = tk.Checkbutton(fenetre, text="Option 1", variable=var1)
check1.pack(pady=5)

check2 = tk.Checkbutton(fenetre, text="Option 2", variable=var2)
check2.pack(pady=5)

check3 = tk.Checkbutton(fenetre, text="Option 3", variable=var3)
check3.pack(pady=5)

def afficher_selection():
 selections = []
 if var1.get():
 selections.append("Option 1")
```

```
if var2.get():
 selections.append("Option 2")
if var3.get():
 selections.append("Option 3")

resultat = ", ".join(selections) if selections else "Aucune sélection"
label_resultat.config(text=f"Sélectionné : {resultat}")
```

```
bouton = tk.Button(fenetre, text="Afficher sélection", command=afficher_selection)
bouton.pack(pady=10)
```

```
label_resultat = tk.Label(fenetre, text="")
label_resultat.pack(pady=10)
```

```
fenetre.mainloop()
...
```

**\*\*Exemple pratique : formulaire avec conditions\*\***

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Inscription newsletter")
```

```
fenetre.geometry("400x300")
```

```
tk.Label(fenetre, text="S'inscrire à la newsletter", font=("Arial", 14, "bold")).pack(pady=10)
```

```
tk.Label(fenetre, text="Email :").pack()

entree_email = tk.Entry(fenetre, width=30)
entree_email.pack(pady=5)

# Checkbuttons pour les préférences
tk.Label(fenetre, text="Recevoir des informations sur :").pack(pady=10)

var_actualites = tk.BooleanVar()
var_promos = tk.BooleanVar()
var_nouveautes = tk.BooleanVar()

tk.Checkbutton(fenetre, text="Actualités", variable=var_actualites).pack()
tk.Checkbutton(fenetre, text="Promotions", variable=var_promos).pack()
tk.Checkbutton(fenetre, text="Nouveautés", variable=var_nouveautes).pack()

# Acceptation des conditions
var_conditions = tk.BooleanVar()
tk.Checkbutton(
    fenetre,
    text="J'accepte les conditions d'utilisation",
    variable=var_conditions
).pack(pady=10)

def valider():
    if not var_conditions.get():
        label_resultat.config(text="Vous devez accepter les conditions", fg="red")
    elif not entree_email.get():
        label_resultat.config(text="Veuillez entrer un email", fg="red")
```

else:

```
    label_resultat.config(text="Inscription réussie !", fg="green")
```

```
tk.Button(fenetre, text="S'inscrire", command=valider, bg="blue", fg="white").pack(pady=10)
```

```
label_resultat = tk.Label(fenetre, text="", font=("Arial", 10))
```

```
label_resultat.pack()
```

```
fenetre.mainloop()
```

```
...
```

Le widget Radiobutton (bouton radio)

Les Radiobuttons permettent de choisir une seule option parmi plusieurs.

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Radiobutton")
```

```
fenetre.geometry("300x250")
```

```
tk.Label(fenetre, text="Quelle est votre couleur préférée ?", font=("Arial",
12)).pack(pady=10)
```

```
Variable partagée par tous les radiobuttons
```

```
couleur_var = tk.StringVar(value="rouge")
```

```
Création des radiobuttons
```

```
tk.Radiobutton(fenetre, text="Rouge", variable=couleur_var, value="rouge").pack()
```

```
tk.Radiobutton(fenetre, text="Vert", variable=couleur_var, value="vert").pack()
```

```
tk.Radiobutton(fenetre, text="Bleu", variable=couleur_var, value="bleu").pack()
```

```
tk.Radiobutton(fenetre, text="Jaune", variable=couleur_var, value="jaune").pack()
```

```
def afficher_choix():
```

```
 choix = couleur_var.get()
```

```
 label_resultat.config(text=f"Vous avez choisi : {choix}")
```

```
tk.Button(fenetre, text="Valider", command=afficher_choix).pack(pady=10)
```

```
label_resultat = tk.Label(fenetre, text="", font=("Arial", 11, "bold"))
```

```
label_resultat.pack(pady=10)
```

```
fenetre.mainloop()
```

```
...
```

```
Exemple avec indicateur de valeur :
```

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Quiz")
```

```
fenetre.geometry("400x300")
```

```
tk.Label(fenetre, text="Quelle est la capitale de la France ?", font=("Arial", 12,
"bold")).pack(pady=20)
```

```
reponse_var = tk.IntVar()
```

```
tk.Radiobutton(fenetre, text="A) Londres", variable=reponse_var, value=1).pack(anchor="w",
padx=50)
```

```
tk.Radiobutton(fenetre, text="B) Paris", variable=reponse_var, value=2).pack(anchor="w",
padx=50)
```

```
tk.Radiobutton(fenetre, text="C) Berlin", variable=reponse_var, value=3).pack(anchor="w",
padx=50)
```

```
tk.Radiobutton(fenetre, text="D) Madrid", variable=reponse_var, value=4).pack(anchor="w",
padx=50)
```

```
def verifier():
```

```
    if reponse_var.get() == 2:
```

```
        label_resultat.config(text="✓ Correct !", fg="green")
```

```
    elif reponse_var.get() == 0:
```

```
        label_resultat.config(text="Veuillez sélectionner une réponse", fg="orange")
```

```
    else:
```

```
        label_resultat.config(text="X Incorrect, essayez encore", fg="red")
```

```
tk.Button(fenetre, text="Vérifier", command=verifier, bg="blue", fg="white").pack(pady=20)
```

```
label_resultat = tk.Label(fenetre, text="", font=("Arial", 12, "bold"))
```

```
label_resultat.pack()
```

```
fenetre.mainloop()
```

```
...
```


Le widget Listbox (liste déroulante)

La Listbox affiche une liste d'éléments sélectionnables.

```
```python
import tkinter as tk

fenetre = tk.Tk()
fenetre.title("Listbox")
fenetre.geometry("300x300")

tk.Label(fenetre, text="Sélectionnez une ville :", font=("Arial", 12)).pack(pady=10)

Création de la Listbox
listbox = tk.Listbox(fenetre, height=8)
listbox.pack(pady=10)

Ajout d'éléments
villes = ["Paris", "Lyon", "Marseille", "Toulouse", "Nice", "Nantes", "Strasbourg", "Bordeaux"]
for ville in villes:
 listbox.insert(tk.END, ville)

def afficher_selection():
 selection = listbox.curselection()
 if selection:
 index = selection[0]
 ville = listbox.get(index)
```

```

 label_resultat.config(text=f"Vous avez sélectionné : {ville}")
 else:
 label_resultat.config(text="Aucune sélection")

tk.Button(fenetre, text="Afficher sélection", command=afficher_selection).pack(pady=5)

label_resultat = tk.Label(fenetre, text="", font=("Arial", 10))
label_resultat.pack(pady=10)

fenetre.mainloop()
...

```

**\*\*Listbox avec sélection multiple :\*\***

```

```python
import tkinter as tk

fenetre = tk.Tk()
fenetre.title("Listbox multiple")
fenetre.geometry("400x350")

tk.Label(fenetre, text="Sélectionnez vos langages préférés :", font=("Arial",
12)).pack(pady=10)

# Listbox avec sélection multiple
listbox = tk.Listbox(fenetre, selectmode=tk.MULTIPLE, height=10)
listbox.pack(pady=10)

```

```
langages = ["Python", "JavaScript", "Java", "C++", "C#", "Ruby", "PHP", "Go", "Swift", "Kotlin"]
```

```
for langage in langages:
```

```
    listBox.insert(tk.END, langage)
```

```
def afficher_selections():
```

```
    selections = listBox.curselection()
```

```
    langages_selectionnees = [listBox.get(i) for i in selections]
```

```
    if langages_selectionnees:
```

```
        texte = "Vos choix : " + ", ".join(langages_selectionnees)
```

```
    else:
```

```
        texte = "Aucune sélection"
```

```
    label_resultat.config(text=texte)
```

```
tk.Button(fenetre, text="Valider", command=afficher_selections, bg="green",  
fg="white").pack(pady=10)
```

```
label_resultat = tk.Label(fenetre, text="", font=("Arial", 10), wraplength=350)
```

```
label_resultat.pack(pady=10)
```

```
fenetre.mainloop()
```

```
...
```

****Méthodes importantes de Listbox :****

- `insert(position, élément)` : ajoute un élément (tk.END pour la fin)

- `get(index)` : récupère l'élément à l'index

- `curselection()` : retourne les indices des éléments sélectionnés

- ``delete(index)`` : supprime un élément
- ``size()`` : retourne le nombre d'éléments

Le widget Scale (curseur)

Le Scale permet de sélectionner une valeur numérique avec un curseur.

```
```python
import tkinter as tk

fenetre = tk.Tk()
fenetre.title("Scale")
fenetre.geometry("400x300")

tk.Label(fenetre, text="Ajustez les valeurs avec les curseurs", font=("Arial", 12,
"bold")).pack(pady=10)

Scale horizontal
tk.Label(fenetre, text="Volume :").pack()
scale1 = tk.Scale(fenetre, from_=0, to=100, orient=tk.HORIZONTAL, length=300)
scale1.set(50)
scale1.pack(pady=10)

Scale vertical
tk.Label(fenetre, text="Luminosité :").pack()
scale2 = tk.Scale(fenetre, from_=0, to=100, orient=tk.VERTICAL, length=150)
scale2.set(75)
scale2.pack(pady=10)
```

```

def afficher_valeurs():
 volume = scale1.get()
 luminosite = scale2.get()
 label_resultat.config(text=f"Volume : {volume}% | Luminosité : {luminosite}%")

tk.Button(fenetre, text="Afficher valeurs", command=afficher_valeurs).pack(pady=10)

label_resultat = tk.Label(fenetre, text="", font=("Arial", 10))
label_resultat.pack()

fenetre.mainloop()
'''

Scale avec mise à jour en temps réel :

```python
import tkinter as tk

fenetre = tk.Tk()
fenetre.title("Scale avec callback")
fenetre.geometry("400x400")

label_couleur = tk.Label(fenetre, text="Ajustez les couleurs RGB", font=("Arial", 14, "bold"))
label_couleur.pack(pady=20)

# Zone de prévisualisation

```

```
zone_couleur = tk.Label(fenetre, text="Aperçu", width=30, height=5, bg="black", fg="white",  
font=("Arial", 14))
```

```
zone_couleur.pack(pady=20)
```

```
def mettre_a_jour_couleur(event=None):
```

```
    rouge = scale_rouge.get()
```

```
    vert = scale_vert.get()
```

```
    bleu = scale_bleu.get()
```

```
    couleur = f"#{rouge:02x}{vert:02x}{bleu:02x}"
```

```
    zone_couleur.config(bg=couleur)
```

```
    label_valeurs.config(text=f"R:{rouge} G:{vert} B:{bleu}")
```

```
# Scales pour RGB
```

```
tk.Label(fenetre, text="Rouge :").pack()
```

```
scale_rouge = tk.Scale(fenetre, from_=0, to=255, orient=tk.HORIZONTAL, length=300,  
command=mettre_a_jour_couleur)
```

```
scale_rouge.set(0)
```

```
scale_rouge.pack()
```

```
tk.Label(fenetre, text="Vert :").pack()
```

```
scale_vert = tk.Scale(fenetre, from_=0, to=255, orient=tk.HORIZONTAL, length=300,  
command=mettre_a_jour_couleur)
```

```
scale_vert.set(0)
```

```
scale_vert.pack()
```

```
tk.Label(fenetre, text="Bleu :").pack()
```

```
scale_bleu = tk.Scale(fenetre, from_=0, to=255, orient=tk.HORIZONTAL, length=300,  
command=mettre_a_jour_couleur)
```

```
scale_bleu.set(0)
```

```
scale_bleu.pack()
```

```
label_valeurs = tk.Label(fenetre, text="R:0 G:0 B:0", font=("Arial", 10))
```

```
label_valeurs.pack(pady=10)
```

```
fenetre.mainloop()
```

```
...
```

Le widget Text (zone de texte multiligne)

Le widget Text permet d'afficher et d'éditer du texte sur plusieurs lignes.

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Widget Text")
```

```
fenetre.geometry("500x400")
```

```
tk.Label(fenetre, text="Éditeur de texte simple", font=("Arial", 14, "bold")).pack(pady=10)
```

```
Zone de texte avec scrollbar
```

```
frame_texte = tk.Frame(fenetre)
```

```
frame_texte.pack(pady=10, padx=20, fill=tk.BOTH, expand=True)
```

```
scrollbar = tk.Scrollbar(frame_texte)
```

```
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
```

```
zone_texte = tk.Text(frame_texte, wrap=tk.WORD, yscrollcommand=scrollbar.set,
font=("Arial", 11))
```

```
zone_texte.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
```

```
scrollbar.config(command=zone_texte.yview)
```

```
Boutons d'action
```

```
frame_boutons = tk.Frame(fenetre)
```

```
frame_boutons.pack(pady=10)
```

```
def recuperer_texte():
```

```
 contenu = zone_texte.get("1.0", tk.END)
```

```
 print("Contenu :")
```

```
 print(contenu)
```

```
 label_info.config(text=f"Texte récupéré ({len(contenu)} caractères)")
```

```
def effacer_texte():
```

```
 zone_texte.delete("1.0", tk.END)
```

```
 label_info.config(text="Texte effacé")
```

```
def inserer_exemple():
```

```
 exemple = "Ceci est un exemple de texte.\nVous pouvez modifier ce texte
librement.\n\nTkinter est puissant !"
```

```
 zone_texte.insert("1.0", exemple)
```

```
 label_info.config(text="Exemple inséré")
```

```
tk.Button(frame_boutons, text="Récupérer texte",
command=recuperer_texte).pack(side=tk.LEFT, padx=5)
```



```
tk.Button(frame_boutons, text="Effacer", command=effacer_texte).pack(side=tk.LEFT,
padx=5)
```

```
tk.Button(frame_boutons, text="Insérer exemple",
command=insérer_exemple).pack(side=tk.LEFT, padx=5)
```

```
label_info = tk.Label(fenetre, text="", font=("Arial", 9), fg="blue")
label_info.pack()
```

```
fenetre.mainloop()
...
```

**\*\*Méthodes importantes de Text :\*\***

- `insert(position, texte)` : insère du texte (position "1.0" = début)
- `get(debut, fin)` : récupère le texte ("1.0" à tk.END pour tout)
- `delete(debut, fin)` : supprime le texte
- La position est au format "ligne.colonne" (ex: "1.0" = ligne 1, colonne 0)

**### Le widget Spinbox (compteur)**

Le Spinbox permet de sélectionner une valeur avec des boutons + et -.

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Spinbox")
```

```
fenetre.geometry("350x250")
```

```
tk.Label(fenetre, text="Configuration", font=("Arial", 14, "bold")).pack(pady=20)
```

```
# Spinbox avec plage de valeurs
```

```
tk.Label(fenetre, text="Âge :").pack()
```

```
spinbox_age = tk.Spinbox(fenetre, from_=0, to=120, width=10)
```

```
spinbox_age.pack(pady=5)
```

```
# Spinbox avec valeurs spécifiques
```

```
tk.Label(fenetre, text="Taille de police :").pack()
```

```
spinbox_police = tk.Spinbox(fenetre, values=(8, 10, 12, 14, 16, 18, 20, 24), width=10)
```

```
spinbox_police.pack(pady=5)
```

```
def afficher_valeurs():
```

```
    age = spinbox_age.get()
```

```
    police = spinbox_police.get()
```

```
    label_resultat.config(text=f"Âge : {age} ans | Police : {police}pt")
```

```
tk.Button(fenetre, text="Valider", command=afficher_valeurs, bg="green",  
fg="white").pack(pady=20)
```

```
label_resultat = tk.Label(fenetre, text="", font=("Arial", 10))
```

```
label_resultat.pack()
```

```
fenetre.mainloop()
```

```
...
```

```
### Exercices Chapitre 5
```

****Exercice 5.1 :**** Créez un formulaire de commande de pizza avec des Checkbuttons pour les ingrédients (tomate, fromage, champignons, jambon, etc.) et affichez le prix total (chaque ingrédient coûte 1,50€).

****Exercice 5.2 :**** Créez un quiz à choix multiples avec 3 questions utilisant des Radiobuttons. Calculez et affichez le score final.

****Exercice 5.3 :**** Créez une application de prise de notes avec un Text pour écrire, et des boutons pour : sauvegarder le texte dans une variable, effacer le texte, et compter le nombre de mots.

****Exercice 5.4 :**** Créez un mélangeur de couleurs avec trois Scale (Rouge, Vert, Bleu) qui changent la couleur de fond de la fenêtre en temps réel.

Chapitre 6 : Organisation et frames

Le widget Frame

Un Frame est un conteneur qui permet d'organiser d'autres widgets. Il est essentiel pour créer des interfaces complexes et structurées.

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Widget Frame")
```

```
fenetre.geometry("400x300")
```

```

Frame avec bordure visible

frame1 = tk.Frame(fenetre, bg="lightblue", relief="solid", bd=2)
frame1.pack(pady=10, padx=10, fill=tk.BOTH, expand=True)

tk.Label(frame1, text="Contenu du Frame 1", bg="lightblue").pack(pady=20)
tk.Button(frame1, text="Bouton 1").pack()

Deuxième frame

frame2 = tk.Frame(fenetre, bg="lightgreen", relief="groove", bd=3)
frame2.pack(pady=10, padx=10, fill=tk.BOTH, expand=True)

tk.Label(frame2, text="Contenu du Frame 2", bg="lightgreen").pack(pady=20)
tk.Button(frame2, text="Bouton 2").pack()

fenetre.mainloop()
...

```

**\*\*Options de relief pour les frames :\*\***

- `"flat"` : plat (par défaut)
- `"raised"` : surélevé
- `"sunken"` : enfoncé
- `"groove"` : rainure
- `"ridge"` : arête
- `"solid"` : bordure solide

**### Organisation avec plusieurs frames**

```

```python

```

```
import tkinter as tk

fenetre = tk.Tk()
fenetre.title("Interface organisée")
fenetre.geometry("600x400")

# Frame supérieur (header)
frame_header = tk.Frame(fenetre, bg="darkblue", height=60)
frame_header.pack(fill=tk.X)

tk.Label(frame_header, text="Mon Application", font=("Arial", 20, "bold"), bg="darkblue",
fg="white").pack(pady=15)

# Frame central avec deux colonnes
frame_central = tk.Frame(fenetre)
frame_central.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

# Colonne gauche (menu)
frame_gauche = tk.Frame(frame_central, bg="lightgray", width=150, relief="ridge", bd=2)
frame_gauche.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 10))
frame_gauche.pack_propagate(False)

tk.Label(frame_gauche, text="Menu", font=("Arial", 12, "bold"),
bg="lightgray").pack(pady=10)

tk.Button(frame_gauche, text="Accueil", width=15).pack(pady=5)
tk.Button(frame_gauche, text="Profil", width=15).pack(pady=5)
tk.Button(frame_gauche, text="Paramètres", width=15).pack(pady=5)
tk.Button(frame_gauche, text="Aide", width=15).pack(pady=5)
```

```

# Colonne droite (contenu principal)

frame_droite = tk.Frame(frame_central, bg="white", relief="sunken", bd=2)

frame_droite.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)


tk.Label(frame_droite, text="Zone de contenu principal", font=("Arial", 14),
bg="white").pack(pady=20)

tk.Label(frame_droite, text="Ici s'affichera le contenu de chaque section", bg="white").pack()


# Frame inférieur (footer)

frame_footer = tk.Frame(fenetre, bg="darkgray", height=30)

frame_footer.pack(fill=tk.X, side=tk.BOTTOM)


tk.Label(frame_footer, text="© 2026 - Mon Application v1.0", bg="darkgray",
fg="white").pack()


fenetre.mainloop()
...

```

Exemple complet : Interface de gestion de tâches

```

``python

import tkinter as tk

from datetime import datetime


fenetre = tk.Tk()

fenetre.title("Gestionnaire de tâches")

fenetre.geometry("700x500")


# Variables

```

```
taches = []
```

```
# Frame header
```

```
frame_header = tk.Frame(fenetre, bg="#2c3e50", height=70)
```

```
frame_header.pack(fill=tk.X)
```

```
frame_header.pack_propagate(False)
```

```
tk.Label(
```

```
    frame_header,
```

```
    text="📝 Gestionnaire de Tâches",
```

```
    font=("Arial", 24, "bold"),
```

```
    bg="#2c3e50",
```

```
    fg="white"
```

```
).pack(pady=20)
```

```
# Frame principal
```

```
frame_principal = tk.Frame(fenetre, bg="#ecf0f1")
```

```
frame_principal.pack(fill=tk.BOTH, expand=True, padx=20, pady=20)
```

```
# Frame gauche : formulaire d'ajout
```

```
frame_formulaire = tk.Frame(frame_principal, bg="white", relief="ridge", bd=2)
```

```
frame_formulaire.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 10))
```

```
tk.Label(
```

```
    frame_formulaire,
```

```
    text="Nouvelle tâche",
```

```
    font=("Arial", 14, "bold"),
```

```
    bg="white"
```

```
).pack(pady=10)
```

```
tk.Label(frame_formulaire, text="Description :", bg="white").pack(pady=(10, 0))
```

```
entree_tache = tk.Entry(frame_formulaire, width=30, font=("Arial", 11))
```

```
entree_tache.pack(pady=5)
```

```
tk.Label(frame_formulaire, text="Priorité :", bg="white").pack(pady=(10, 0))
```

```
priorite_var = tk.StringVar(value="Moyenne")
```

```
for priorite in ["Haute", "Moyenne", "Basse"]:
```

```
    tk.Radiobutton(
        frame_formulaire,
        text=priorite,
        variable=priorite_var,
        value=priorite,
        bg="white"
    ).pack()
```

```
def ajouter_tache():
```

```
    description = entree_tache.get()
```

```
    if description:
```

```
        priorite = priorite_var.get()
```

```
        heure = datetime.now().strftime("%H:%M")
```

```
        tache = f"[{heure}] {description} [{priorite}]"
```

```
        taches.append(tache)
```

```
        listbox_taches.insert(tk.END, tache)
```

```
        entree_tache.delete(0, tk.END)
```

```
        mettre_a_jour_compteur()
```



```
tk.Button(  
    frame_formulaire,  
    text="Ajouter",  
    command=ajouter_tache,  
    bg="#27ae60",  
    fg="white",  
    font=("Arial", 12, "bold"),  
    width=20,  
    pady=5  
).pack(pady=20)
```

```
def supprimer_tache():  
    selection = listbox_taches.curselection()  
    if selection:  
        index = selection[0]  
        listbox_taches.delete(index)  
        taches.pop(index)  
        mettre_a_jour_compteur()
```

```
tk.Button(  
    frame_formulaire,  
    text="Supprimer sélection",  
    command=supprimer_tache,  
    bg="#e74c3c",  
    fg="white",  
    width=20  
).pack(pady=5)
```

```
def effacer_tout():  
    listBox_taches.delete(0, tk.END)  
    taches.clear()  
    mettre_a_jour_compteur()
```

```
tk.Button(  
    frame_formulaire,  
    text="Tout effacer",  
    command=effacer_tout,  
    bg="#95a5a6",  
    fg="white",  
    width=20  
) .pack(pady=5)
```

```
# Frame droite : liste des tâches
```

```
frame_liste = tk.Frame(frame_principal, bg="white", relief="ridge", bd=2)  
frame_liste.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
```

```
tk.Label(  
    frame_liste,  
    text="Mes tâches",  
    font=("Arial", 14, "bold"),  
    bg="white"  
) .pack(pady=10)
```

```
# Listbox avec scrollbar
```

```
frame_listbox = tk.Frame(frame_liste, bg="white")  
frame_listbox.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
```

```
scrollbar = tk.Scrollbar(frame_listbox)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
```

```
listbox_taches = tk.Listbox(
    frame_listbox,
    yscrollcommand=scrollbar.set,
    font=("Courier", 10),
    selectmode=tk.SINGLE
)
listbox_taches.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
scrollbar.config(command=listbox_taches.yview)
```

```
def mettre_a_jour_compteur():
    nombre = len(taches)
    label_compteur.config(text=f"Total : {nombre} tâche(s)")
```

```
label_compteur = tk.Label(frame_liste, text="Total : 0 tâche(s)", bg="white", font=("Arial",
10))
label_compteur.pack(pady=10)
```

```
fenetre.mainloop()
...
```

Frames imbriqués pour layouts complexes

```
```python
import tkinter as tk
```

```
fenetre = tk.Tk()

fenetre.title("Frames imbriqués")

fenetre.geometry("800x600")

Frame principal

frame_principal = tk.Frame(fenetre, bg="white")

frame_principal.pack(fill=tk.BOTH, expand=True)

Trois sections horizontales

frame_top = tk.Frame(frame_principal, bg="lightblue", height=100)

frame_top.pack(fill=tk.X)

tk.Label(frame_top, text="Section supérieure", font=("Arial", 16),
bg="lightblue").pack(pady=30)

frame_middle = tk.Frame(frame_principal, bg="white")

frame_middle.pack(fill=tk.BOTH, expand=True)

frame_bottom = tk.Frame(frame_principal, bg="lightgray", height=50)

frame_bottom.pack(fill=tk.X)

tk.Label(frame_bottom, text="Section inférieure", bg="lightgray").pack(pady=15)

Dans la section du milieu : trois colonnes

frame_left = tk.Frame(frame_middle, bg="lightyellow", width=200)

frame_left.pack(side=tk.LEFT, fill=tk.Y)

frame_left.pack_propagate(False)

tk.Label(frame_left, text="Colonne\nGauche", bg="lightyellow", font=("Arial",
12)).pack(pady=20)
```

```

frame_center = tk.Frame(frame_middle, bg="white")
frame_center.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

Dans la colonne centrale : deux lignes
frame_center_top = tk.Frame(frame_center, bg="lightgreen", height=200)
frame_center_top.pack(fill=tk.X, pady=10, padx=10)
tk.Label(frame_center_top, text="Centre Haut", bg="lightgreen", font=("Arial",
14)).pack(pady=80)

frame_center_bottom = tk.Frame(frame_center, bg="lightcoral")
frame_center_bottom.pack(fill=tk.BOTH, expand=True, pady=(0, 10), padx=10)
tk.Label(frame_center_bottom, text="Centre Bas", bg="lightcoral", font=("Arial",
14)).pack(pady=50)

frame_right = tk.Frame(frame_middle, bg="lightblue", width=200)
frame_right.pack(side=tk.RIGHT, fill=tk.Y)
frame_right.pack_propagate(False)
tk.Label(frame_right, text="Colonne\nDroite", bg="lightblue", font=("Arial",
12)).pack(pady=20)

fenetre.mainloop()
...

```

### LabelFrame : Frame avec titre

Le LabelFrame est un Frame avec un titre intégré, utile pour regrouper des éléments liés.

```

```python
import tkinter as tk

```

```
fenetre = tk.Tk()

fenetre.title("LabelFrame")

fenetre.geometry("500x400")


# LabelFrame pour informations personnelles

lf_perso = tk.LabelFrame(fenetre, text="Informations personnelles", font=("Arial", 12,
"bold"), padx=20, pady=20)

lf_perso.pack(padx=20, pady=10, fill=tk.BOTH)


tk.Label(lf_perso, text="Nom :").grid(row=0, column=0, sticky="e", pady=5)
tk.Entry(lf_perso, width=30).grid(row=0, column=1, pady=5)


tk.Label(lf_perso, text="Prénom :").grid(row=1, column=0, sticky="e", pady=5)
tk.Entry(lf_perso, width=30).grid(row=1, column=1, pady=5)


tk.Label(lf_perso, text="Date de naissance :").grid(row=2, column=0, sticky="e", pady=5)
tk.Entry(lf_perso, width=30).grid(row=2, column=1, pady=5)


# LabelFrame pour adresse

lf_adresse = tk.LabelFrame(fenetre, text="Adresse", font=("Arial", 12, "bold"), padx=20,
pady=20)

lf_adresse.pack(padx=20, pady=10, fill=tk.BOTH)


tk.Label(lf_adresse, text="Rue :").grid(row=0, column=0, sticky="e", pady=5)
tk.Entry(lf_adresse, width=30).grid(row=0, column=1, pady=5)


tk.Label(lf_adresse, text="Ville :").grid(row=1, column=0, sticky="e", pady=5)
tk.Entry(lf_adresse, width=30).grid(row=1, column=1, pady=5)
```

```
tk.Label(lf_adresse, text="Code postal :").grid(row=2, column=0, sticky="e", pady=5)
```

```
tk.Entry(lf_adresse, width=30).grid(row=2, column=1, pady=5)
```

```
tk.Button(fenetre, text="Valider", bg="green", fg="white", font=("Arial", 12),  
width=15).pack(pady=20)
```

```
fenetre.mainloop()
```

```
...
```

Exercices Chapitre 6

****Exercice 6.1 : **** Créez une calculatrice avec frames : un frame pour l'affichage en haut, un frame pour les boutons numéros au milieu, et un frame pour les opérateurs sur le côté.

****Exercice 6.2 : **** Créez une interface de blog avec trois frames : header (titre du blog), sidebar gauche (catégories), et zone de contenu principale (articles).

****Exercice 6.3 : **** Créez un formulaire d'inscription complet en utilisant des LabelFrame pour regrouper : Identité, Coordonnées, Préférences, et Conditions d'utilisation.

```
---
```

Chapitre 7 : Menus et boîtes de dialogue

Le widget Menu

Les menus permettent de créer des barres de menus avec des options déroulantes.

```
```python
import tkinter as tk

fenetre = tk.Tk()
fenetre.title("Menu simple")
fenetre.geometry("400x300")

Créer la barre de menu
barre_menu = tk.Menu(fenetre)
fenetre.config(menu=barre_menu)

Menu Fichier
menu_fichier = tk.Menu(barre_menu, tearoff=0)
barre_menu.add_cascade(label="Fichier", menu=menu_fichier)

def nouveau():
 print("Nouveau fichier")

def ouvrir():
 print("Ouvrir fichier")

def sauvegarder():
 print("Sauvegarder fichier")

def quitter():
 fenetre.quit()

menu_fichier.add_command(label="Nouveau", command=nouveau)
```



```
menu_fichier.add_command(label="Ouvrir", command=ouvrir)
menu_fichier.add_command(label="Sauvegarder", command=sauvegarder)
menu_fichier.add_separator()
menu_fichier.add_command(label="Quitter", command=quitter)
```

#### # Menu Édition

```
menu_edition = tk.Menu(barre_menu, tearoff=0)
barre_menu.add_cascade(label="Édition", menu=menu_edition)
```

```
menu_edition.add_command(label="Copier")
menu_edition.add_command(label="Coller")
menu_edition.add_command(label="Couper")
```

#### # Menu Aide

```
menu_aide = tk.Menu(barre_menu, tearoff=0)
barre_menu.add_cascade(label="Aide", menu=menu_aide)
```

```
menu_aide.add_command(label="Documentation")
menu_aide.add_command(label="À propos")
```

```
fenetre.mainloop()
```

```
...
```

#### **\*\*Explications : \*\***

- `tearoff=0` : empêche le menu d'être détachable
- `add\_cascade()` : ajoute un menu déroulant
- `add\_command()` : ajoute une option de menu
- `add\_separator()` : ajoute une ligne de séparation

### Menu avec sous-menus

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Menu avec sous-menus")
```

```
fenetre.geometry("500x400")
```

```
barre_menu = tk.Menu(fenetre)
```

```
fenetre.config(menu=barre_menu)
```

```
# Menu principal
```

```
menu_fichier = tk.Menu(barre_menu, tearoff=0)
```

```
barre_menu.add_cascade(label="Fichier", menu=menu_fichier)
```

```
# Sous-menu Export
```

```
sous_menu_export = tk.Menu(menu_fichier, tearoff=0)
```

```
sous_menu_export.add_command(label="Export PDF")
```

```
sous_menu_export.add_command(label="Export CSV")
```

```
sous_menu_export.add_command(label="Export Excel")
```

```
menu_fichier.add_command(label="Nouveau")
```

```
menu_fichier.add_command(label="Ouvrir")
```

```
menu_fichier.add_separator()
```

```
menu_fichier.add_cascade(label="Exporter", menu=sous_menu_export)
```

```
menu_fichier.add_separator()
```

```
menu_fichier.add_command(label="Quitter", command=fenetre.quit)
```

```
fenetre.mainloop()
```

```
...
```

```
### Menu contextuel (clic droit)
```

```
```python
```

```
import tkinter as tk
```

```
fenetre = tk.Tk()
```

```
fenetre.title("Menu contextuel")
```

```
fenetre.geometry("400x300")
```

```
label = tk.Label(fenetre, text="Faites un clic droit sur cette fenêtre", font=("Arial", 14))
```

```
label.pack(pady=100)
```

```
Créer le menu contextuel
```

```
menu_contextuel = tk.Menu(fenetre, tearoff=0)
```

```
def copier():
```

```
 print("Copier")
```

```
def coller():
```

```
 print("Coller")
```

```
def supprimer():
```

```
 print("Supprimer")
```

```
menu_contextuel.add_command(label="Copier", command=copier)
menu_contextuel.add_command(label="Coller", command=coller)
menu_contextuel.add_separator()
menu_contextuel.add_command(label="Supprimer", command=supprimer)
```

```
Fonction pour afficher le menu
```

```
def afficher_menu_contextuel(event):
 menu_contextuel.post(event.x_root, event.y_root)
```

```
Lier le clic droit à la fonction
```

```
fenetre.bind("<Button-3>", afficher_menu_contextuel)
```

```
fenetre.mainloop()
```

```
...
```

```
MessageBox : boîtes de dialogue
```

Le module `messagebox` fournit des boîtes de dialogue prédéfinies.

```
```python
```

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
fenetre = tk.Tk()
```

```
fenetre.title("MessageBox")
```

```
fenetre.geometry("400x400")
```

```
tk.Label(fenetre, text="Exemples de boîtes de dialogue", font=("Arial", 14, "bold")).pack(pady=20)
```

```
def info():
```

```
    messagebox.showinfo("Information", "Ceci est un message d'information")
```

```
def avertissement():
```

```
    messagebox.showwarning("Avertissement", "Attention ! Ceci est un avertissement")
```

```
def erreur():
```

```
    messagebox.showerror("Erreur", "Une erreur s'est produite")
```

```
def question():
```

```
    reponse = messagebox.askquestion("Question", "Voulez-vous continuer ?")
```

```
    if reponse == "yes":
```

```
        print("L'utilisateur a dit oui")
```

```
    else:
```

```
        print("L'utilisateur a dit non")
```

```
def okcancel():
```

```
    reponse = messagebox.askokcancel("Confirmation", "Êtes-vous sûr de vouloir supprimer ?")
```

```
    if reponse:
```

```
        print("Suppression confirmée")
```

```
    else:
```

```
        print("Suppression annulée")
```

```
def yesnocancel():
```

```
reponse = messagebox.askyesnocancel("Sauvegarder", "Voulez-vous sauvegarder les  
modifications ?")
```

```
if reponse == True:
```

```
    print("Sauvegarder")
```

```
elif reponse == False:
```

```
    print("Ne pas sauvegarder")
```

```
else:
```

```
    print("Annuler")
```

```
tk.Button(fenetre, text="Information", command=info, width=20).pack(pady=5)
```

```
tk.Button(fenetre, text="Avertissement", command=avertissement, width=20).pack(pady=5)
```

```
tk.Button(fenetre, text="Erreur", command=erreur, width=20).pack(pady=5)
```

```
tk.Button(fenetre, text="Question", command=question, width=20).pack(pady=5)
```

```
tk.Button(fenetre, text="OK/Annuler", command=okcancel, width=20).pack(pady=5)
```

```
tk.Button(fenetre, text="Oui/Non/Annuler", command=yesnocancel,  
width=20).pack(pady=5)
```

```
fenetre.mainloop()
```

```
'''
```

```
**
```